

INTERSYSTEMS PT DEVELOPER COMMUNITY 2026 — PROGRAMMING CONTEST

IRIS Production Guardian

An observability, incident-investigation and documentation-query copilot for InterSystems IRIS administrators and developers — built entirely on InterSystems IRIS.

Complete project documentation — version 2026-09-13

Repository: github.com/sergiofsq/iris-production-guardian

Author/owner: Sergio Fernandes · AI-assisted development (Claude Code)

Contents

- 1 Executive summary
- 2 Overview and goals
- 3 Technology constraints
- 4 Architecture
- 5 Core modules
- 6 Bonus features
- 7 Installation and configuration
- 8 Testing and evidence
- 9 Security and secrets audit
- 10 AI-assisted development methodology
- 11 Timeline and project status
- 12 References

1. Executive summary

IRIS Production Guardian is an application built entirely on InterSystems IRIS (ObjectScript, native interoperability, native Vector Search and Foreign Tables — no external database) that observes an interoperability Production in real time, investigates incidents with real evidence collected automatically, and answers questions about the project's documentation with source citation and calibrated abstention when there isn't enough information.

The project was built as a submission for the InterSystems PT Developer Community 2026 Programming Contest, with a submission deadline of 2026-09-21. This document consolidates, in one place, what was built, how it was verified, and what's still open — every claim here corresponds to a real test that was actually run, not an assumption of how the system should behave.

State as of 2026-09-13: the three MVP modules (Monitor, Investigator, RAG Assistant) and all four bonus items (Business Rules, hybrid search, public API access, multi-model) are implemented and tested. The four formal paths required by the internal Definition of Done (happy path, destination failure, no-data, model unavailability) were tested live. A clean-checkout install was verified — six real documentation gaps were found and fixed in the process. What remains is delivery content (demo video and community article), not engineering.

2. Overview and goals

The application solves a real operational problem: an IRIS Production administrator needs to know, at any moment, (a) what's healthy and what isn't, (b) why a specific component is having trouble — with evidence, not guesswork — and (c) how to fix it, grounded in the project's actual documentation, without depending on tribal knowledge locked in one person's head.

2.1 Production Monitor

A dashboard showing the real state of each Production component (Service, Process, Operation): healthy, degraded, unavailable or unknown, with queue count, processed-message count and recent errors, refreshed every 10 seconds. It persists a minimal time series and detects "stuck" hosts (a positive queue that never drains).

2.2 AI Incident Investigator

Given a component and a time window, it gathers real evidence (current health, real-error events, relevant RAG documentation) and asks an AI model for a structured summary: observations, hypotheses explicitly separated from observed fact, evidence gaps, and suggested next steps. It is read-only — no remediation is ever applied automatically.

2.3 RAG Assistant

Answers questions about the project using an indexed documentation corpus (hybrid vector + lexical retrieval), always citing the source, and refusing to answer ("I don't know") when similarity to the corpus is low, instead of inventing an answer. Available with two independent AI providers for the generation step (see section 6.4).

3. Technology constraints (non-negotiable)

- **InterSystems ObjectScript (COS) first** for business logic, interoperability and orchestration — no external application framework.
- **InterSystems IRIS is the only database.** No PostgreSQL, SQLite, MongoDB, Redis, or external vector database — native IRIS Vector Search and Foreign Tables cover that need.
- **Python only where COS provably cannot do the job**, with the limitation documented case by case. In practice, the final project needed zero Python — everything runs on pure COS.
- **Web UI:** plain HTML/CSS plus the minimum presentation JavaScript (loading spinners, theme/language toggles); all business rules stay in the COS backend.

4. Architecture

```
Web UI: Monitor | Investigator | RAG Assistant (Gemini) | RAG Assistant (Groq)
      |
Application layer: queries, investigation, hybrid retrieval
      |
InterSystems IRIS: persistence and interoperability
      |           |           |
COS Production   Events/metrics   Documents/vector indexes
      |                               (native Vector Search + iFind)
Service -> Process -> Operation
      |           (severity-based routing via a Business Rule)
      +--> default destination / priority destination
      |
Second, independent Production: PublicHealth (bonus, real public API)
```

Two independent Productions run in the same namespace (`GUARDIAN`): the main one (synthetic incidents, Service→Process→Operation) and a second one dedicated to the public-API bonus, polling a real external service that needs no authentication key.

5. Core modules — technical detail

5.1 COS Production

Class	Role
<code>Guardian.Messages.IncidentEvent</code>	Message (<code>Ens.Request</code>) representing a synthetic incident
<code>Guardian.Service.FileIncidentService</code>	Business Service, <code>EnsLib.File.InboundAdapter</code> , reads incidents from <code>.txt</code> files
<code>Guardian.Process.IncidentRouterProcess</code>	Business Process in plain COS code (not BPL)
<code>Guardian.Operation.FileOutputOperation</code>	Business Operation, default destination (LOW severity)
<code>Guardian.Operation.PriorityOutputOperation</code>	Business Operation, priority destination (HIGH/CRITICAL/MEDIUM)
<code>Guardian.Production.GuardianProduction</code>	Wires the hosts above together

Real, reproducible failure/recovery scenario: destination directory write permission removed via `chmod` (a real OS-level failure, not simulated), a real `#5005` error captured, the framework's `FailureTimeout` observed actually expiring, automatic recovery confirmed for new traffic, manual resend via `Ens.MessageHeader.ResendMessage` for the message that failed during the outage window.

5.2 Production Monitor

`Guardian.Monitor.StatusCollector` queries real `Ens.*` classes (never a fixed value) for each host's health/queue/messages/errors; `Guardian.API.MonitorAPI` exposes it as JSON; `Guardian.UI.MonitorPage` renders the cards and persists samples into `Guardian_Monitor.HealthSample` on every load.

5.3 AI Incident Investigator

`Guardian.Investigator.EvidenceCollector` gathers current health, real events for the window (via `$System.Status.GetErrorText()`, not the raw `%Status`) and relevant documentation retrieved from the RAG corpus. `Guardian.Investigator.Analyzer` builds the prompt and calls the AI model, explicitly separating observation from hypothesis. Every investigation is persisted to `Guardian_Investigator.Investigation`.

5.4 RAG Assistant

Schema (`Guardian.RAG.Schema`): `Guardian_RAG.Document` and `Guardian_RAG.Chunk` tables, with a native `VECTOR(DOUBLE, 768)` column. Ingestion (`Guardian.RAG.Ingestion`) chunks text (800

characters, 150-character overlap) and computes real embeddings via the provider's API. Hybrid retrieval (`Guardian.RAG.Query.Retrieve`): vector search (cosine similarity, `VECTOR_COSINE`) fused via Reciprocal Rank Fusion with lexical search (a native `%iFind.Index.Basic` index). Generation (`Guardian.RAG.Query.Ask`) requires source citation in the prompt and abstains when the best similarity falls below a calibrated threshold (0.58) — the generation step accepts any client implementing `Generate(prompt)`, so swapping providers never duplicates retrieval logic.

6. Bonus features

6.1 Business Rules BONUS +2

`Guardian.Rule.IncidentRoutingRule` (`Ens.Rule.Definition`) decides the incident's destination by severity, editable visually in the Management Portal's Rule Editor, with no recompilation of the Process. HIGH/CRITICAL/MEDIUM severities go to `PriorityOutputOperation` (a separate physical directory, `/durable/guardian/out_priority`); everything else goes to `FileOutputOperation` .

6.2 Hybrid search BONUS +3

RAG retrieval combines pure vector similarity with a native IRIS full-text index (iFind), fused via Reciprocal Rank Fusion (k=60). The abstention threshold still relies exclusively on pure vector similarity, so it doesn't invalidate the calibration that was already validated.

6.3 Real public API access — PublicHealth BONUS +2

A second, independent Production (`Guardian.Production.PublicHealthProduction`) doing real polling of `disease.sh/v3/covid-19/all` (a public, keyless API), via a custom `Ens.InboundAdapter` . Honest resilience pattern: success writes a "live" row; failure falls back to the last known real value, labeled as cache with a computed age and the HTTP status that caused the fallback; with no cache available it marks unavailability — it never fabricates a number. Tested with a real forced failure (endpoint pointed at an invalid path, a real HTTP 404).

6.4 Multi-model — a second, real AI provider BONUS

A genuinely wired second generation provider: **Groq** (`Guardian.RAG.GroqClient` , model `openai/gpt-oss-120b` , an OpenAI-compatible Chat Completions API), free tier, no credit card required. Retrieval/embeddings stay on the primary provider (Gemini) — Groq offers no embeddings endpoint on its free tier, confirmed before deciding the architecture, not assumed. Tested end-to-end through the real page: investigation → suggested question → a real, cited answer generated by Groq.

7. Installation and configuration

Full, verified steps live in the repository's `README.md` ("Running it locally"). Summary:

1. Start IRIS Community Edition in Docker (image `intersystems/iris-community:latest-cd` — not the `-arm64` variant, whose bundled Community license is expired).
2. Create the `GUARDIAN` namespace / `GUARDIANDB` database with interoperability enabled, and an administrative user.
3. Compile the classes under `src/Guardian/` and create the SQL tables by calling the 4 `*.Schema.Setup()` classes (RAG, Monitor, Investigator, PublicHealth) — this does not happen automatically at compile time.
4. Create the file adapters' runtime directories and start the Production.
5. Serve the static assets (logo, CSS, spinner) by copying them into the CSP application's physical directory.
6. Configure outbound SSL and the AI provider credentials (Gemini and/or Groq) in `Ens.Config.Credentials` — never in source code.
7. Ingest the RAG documentation corpus.

A clean-checkout install was verified live on 2026-09-13, in a throwaway Docker container and volume, with no shortcut from any already-configured environment. Six real documentation gaps were found and fixed in that process — see section 8.5.

8. Testing and evidence

The project's internal Definition of Done requires real, tested evidence for every claim — no plans, stubs or mocks. The four formal paths below were tested live on 2026-09-13.

8.1 Happy path

A LOW-severity message dropped as a file flowed Service → Process → Operation, output file generated in ~10 seconds. The Monitor was confirmed via a real HTTP request (real user login): state `Running`, all 4 hosts `healthy`.

8.2 Destination failure

Destination directory write permission removed via a real `chmod`. Real error captured: `ERROR #5005: Cannot open file '/durable/guardian/out_priority/incident_INC-FASE5-004.txt', wrapped in <Ens>ErrFailureTimeout`. The component's state changed to `degraded` in the Investigator. Permission fixed: new traffic delivered automatically; the message that had already failed is not redelivered on its own (framework behavior) — resent manually via `Ens.MessageHeader.ResendMessage`, confirmed at the destination.

Real finding during this test: the documented script used CRITICAL severity pointing at the old destination directory — made stale by the Business Rules bonus (section 6.1), which now routes that severity to a different destination. Breaking the old directory caused no failure at all. The script was corrected to target the right directory.

8.3 No data

RAG Assistant, asked an irrelevant question ("what's the chocolate cake recipe?"): answered "I don't know" instead of inventing, even with a recent investigation in context. AI Incident Investigator, a component with 65 seconds of zero traffic, 1-minute window: correctly reported "0 events recorded," with no fabricated activity.

8.4 AI model unavailability

Formalized as a deterministic test case (not dependent on waiting for a real rate limit to hit by chance): the real Gemini credential was temporarily swapped for an invalid value in a single session, a real error was captured (`HTTP 400: API key not valid`), the original credential was restored before the session closed, and confirmed working again with a real call afterward.

8.5 Clean-checkout install

A brand-new Docker container and volume, following only the README, with no shortcut from any prior manual setup. Six real gaps found and fixed: (1) a stale project-status table in the README; (2) "interoperability enabled" had no documented scriptable equivalent; (3) the adapters' runtime directories are never created automatically; (4) the static-assets step was incomplete — one file (the loading

spinner) had never been committed to the repository at all; (5) no mention of SSL config or AI credentials; (6) the 4 SQL-schema setup calls were undocumented anywhere. The full app (Monitor, Investigator, RAG with a real ingested document and a real citation) was confirmed working end-to-end in the fresh environment before it was torn down.

9. Security and secrets audit

A full audit was run on 2026-09-13, covering not just the repository's current state but **its entire git history** — relevant because the repository is public, and a secret removed later would still be exposed in older commits.

- No real API key or password found in any commit ever made (pattern search across the whole history, not just the current HEAD).
- Credentials (Gemini, Groq) are always read at runtime from `Ens.Config.Credentials`, never in source code — explicit placeholders (`<YOUR_REAL_KEY_HERE>`) throughout the setup documentation.
- A personal folder containing a plaintext password was confirmed **never tracked** by git since the first commit.
- A privacy finding (not a security one): screenshots used in the user manual exposed the owner's full browser bookmark bar. Fixed by cropping the source images and the images already embedded in the `.docx`, without changing anything already visible when reading the document normally.

10. AI-assisted development methodology

The project was built with AI assistance (Claude Code), under a strict anti-hallucination rule set: never claim something was tested without a real, observed run; never invent classes, methods, or results; log every open technical question as `VERIFY_REQUIRED` until there's real evidence; always distinguish planned, implemented, tested, and demonstrated.

The real prompt sequence that reproduces this application — reconstructed by reverse-engineering from the code, the commits, and the planning documents where the literal original prompt wasn't preserved — is published under `entregaveis/Prompts/` in the repository, one file per phase/bonus, each pairing the prompt with the real errors found and corrected at that step (not an idealized version without the friction that actually happened). Real examples of AI error/hallucination correction documented throughout the project:

- An AI model name outdated in the agent's own knowledge, corrected by the API's own error response (happened twice, with two different providers).
- Use of a non-existent IRIS API (`%GetLastIdentity()`), fixed by using the real INSERT resultset's `%ROWID` instead.
- A false UTF-8 encoding alarm investigated and dismissed with real byte-level evidence (a hexdump), instead of "fixing" code that was already correct.
- An assumption about how to enable interoperability via the GUI with no scriptable equivalent — resolved by real trial and error against the IRIS class dictionary, not by external documentation.

11. Timeline and project status

Period	Work	Status as of 2026-09-13
Sep 8–10	Phase 0 — environment, architecture	DONE
Sep 11–14	Phase 1 + Phase 2 — COS Production, Monitor	DONE, AHEAD OF SCHEDULE
Sep 15–17	Phase 3 + Phase 4 — RAG, Investigator	DONE, AHEAD OF SCHEDULE
Sep 18–19	Phase 5 — hardening, testing, clean install	DONE, A WEEK AHEAD
Sep 20	Video, article, final audit	PENDING
Sep 21	Submission	PENDING

Open items

- Demo video — not started.
- Community article (required for submission, in Portuguese) — not started.
- Reproducible re-ingestion for 5 of the 12 RAG corpus documents (external pages) — no script recorded yet.
- Two `VERIFY_REQUIRED` items that depend on the contest organizers: the official submission deadline/timezone; the interpretation of the bonus-points cap.

Honest assessment: every engineering item in the internal Definition of Done is closed with real evidence. What remains is delivery content (video and article), not development — and the project is ahead of its original schedule.

12. References

- Repository: `github.com/sergiofsq/iris-production-guardian`
- `docs/planejamento/00_MASTER_PLAN.md` — master plan and chronological pendency log (Portuguese)
- `docs/planejamento/01 - 06_*.md` — per-phase documents, with dated evidence (Portuguese)
- `docs/planejamento/07_SCORECARD_EVIDENCIAS.md` — evidence scorecard against contest criteria (Portuguese)
- `docs/experiments/01_falha_recuperacao_producao.md` — reproducible failure/recovery script (Portuguese)
- `entregaveis/Prompts/` — AI-assisted development prompt sequence
- `entregaveis/Manual_IRIS_Production_Guardian.pdf` / `IRIS_Production_Guardian_User_Manual_EN.docx` — application user manual